

Model Evaluation and Guardrail Validation

Chapter 07 · Episode 01 · Domain 2.2 — Testing What Attackers Test

Accuracy Is Not Security

Accuracy = average performance

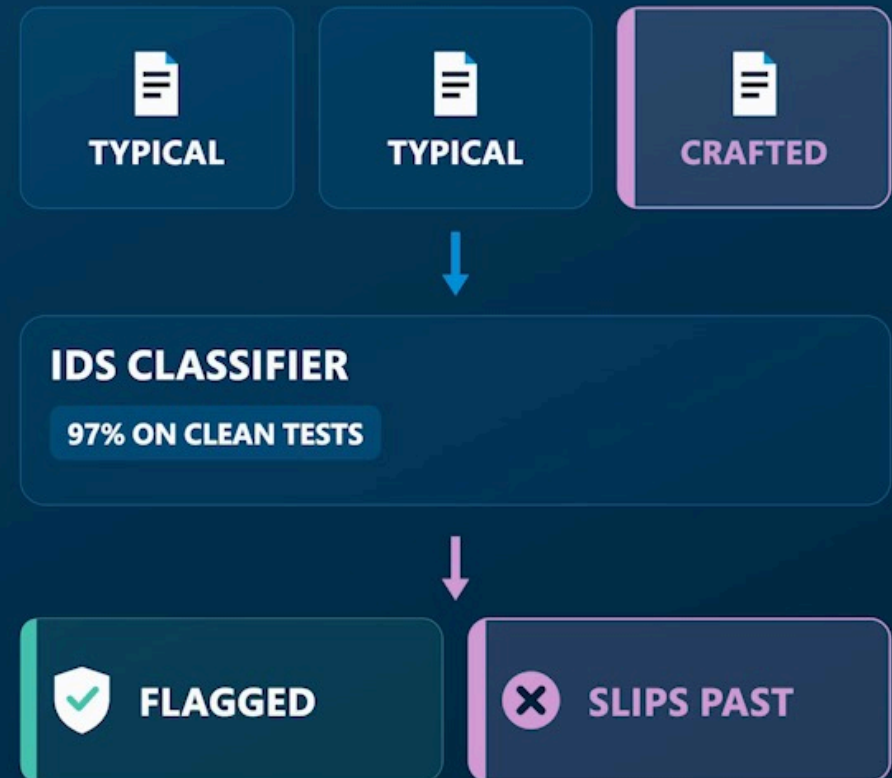
Security = adversarial resilience



97% Accurate — Still Evadable

Clean test set — no adversarial traffic
Crafted inputs slip past boundaries

ONE IDS MODEL · 97% ACCURATE



Accuracy averages typical traffic — the attacker only sends the crafted flow.

Asymmetric Consequences

FALSE POSITIVE

User inconvenienced

Noise you can tune

FALSE NEGATIVE

Breach enabled

Watch high-severity FN rate

Dataset Discipline



HOLDOUT

Never seen in training —
honest baseline



CROSS-VALIDATE

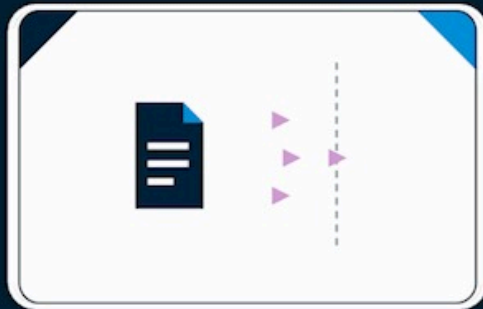
Rotating subsets catch overfitting



BIAS

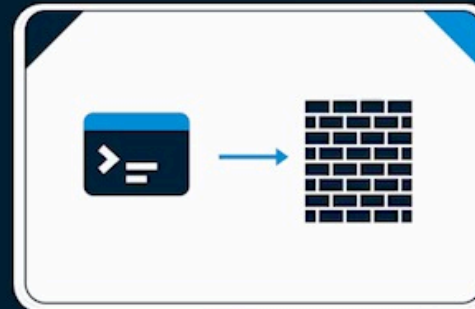
Lab-clean test data
fails in production

Adversarial Robustness Testing



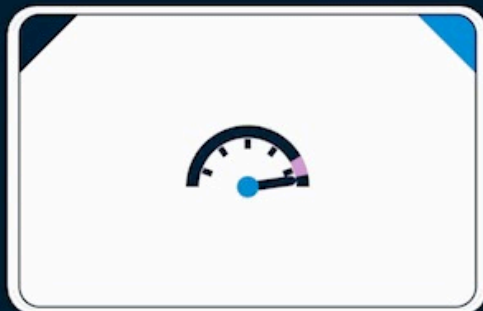
EXAMPLES

Crafted perturbations
probe boundaries



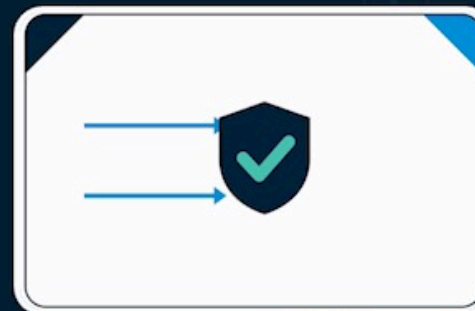
RED TEAM

Injection · extraction · jailbreaks



STRESS

Out-of-distribution extremes



GUARDRAILS

Do constraints hold under attack?

More Than Ethics

BIAS

Blind spots become evasion paths

FAIRNESS

Inconsistency invites probing

EXPLAIN

Trace attack vs drift vs training

Operational properties — not optional extras

DETECTION BY MALWARE FAMILY

RANSOMWARE

CAUGHT

TROJANS

CAUGHT

BOTNETS

CAUGHT

NOVEL LOADERS

MISSED



THE EVASION PATH

A biased blind spot is predictable — attackers find it before you do.



Continuous Evaluation

1. Pre-deployment — baseline robustness gates
2. Post-deployment — drift and anomaly watch
3. Regression test after every update
4. Cycle: evaluate, deploy, monitor, retest

1 CLEAN ACCURACY
Holdout

2 FALSE NEGATIVE
Rate

3 ADVERSARIAL
Robustness

4 BIAS PARITY
Score

5 GUARDRAIL
Hold Rate

HOW TO RUN

WINDOWS

```
python model_robustness_evaluator.py
```

MACOS/LINUX

```
python3 model_robustness_evaluator.py
```

TRY IT Set GUARDRAIL_ENABLED=False. Guardrail hold rate drops to ~12% while clean accuracy stays at 91%. This is the accuracy/security gap in action.

`model_robustness_evaluator.py`

multi-metric evaluation suite with PASS/WARN/FAIL ratings

Demo: Model Robustness Evaluator

Accuracy · adversarial · bias · guardrail hold — PASS/WARN/FAIL

Episode Takeaways

Accuracy is average performance — security is adversarial resilience

Holdout sets and cross-validation keep evaluation honest

Bias is a security vulnerability with a known evasion path

Evaluate continuously — before, during, and after every update

Designing Model Guardrails

Chapter 07 · Episode 02 · Domain 2.2 — Capability vs Permission

Capability vs Permission

CAPABILITY

What the model CAN do

Property of the model

PERMISSION

What it's ALLOWED to do

Property of the deployment

Fix the Enforcement Layer

Guardrail failure $\neq$ training failure

Improve enforcement logic, not weights

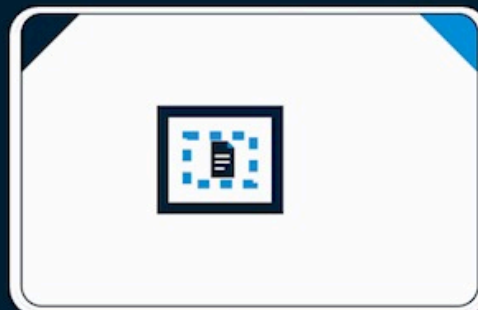


Input Guardrails — First Line



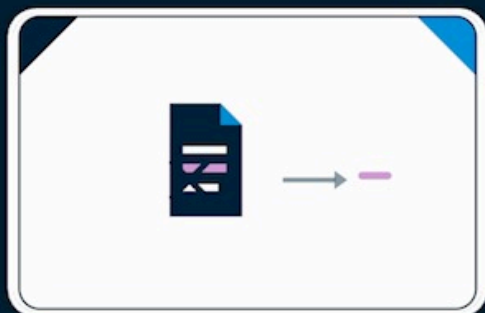
VALIDATE

Filter injection +
policy violations



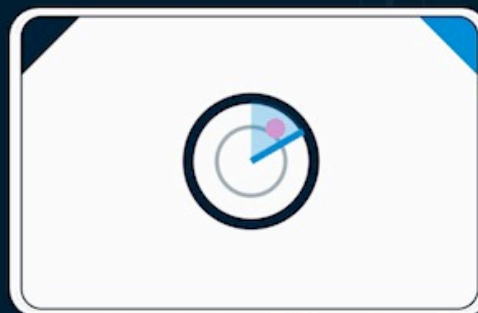
BOUND

Cap context —
less injection surface



SANITIZE

Strip override and
jailbreak framing



DETECT

Catch structural
prompt manipulation

Output Guardrails — Last Line

MODERATE

Scan for prohibited content

POLICY

Business rules bound responses

CONFIDENCE

Low certainty → human review

SCHEMA

Reject off-format responses

Schema limits exfiltration even when injection lands

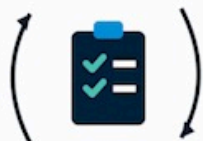




Agent Guardrails

1. Least-privilege tool access
2. Restrict APIs, conditions, parameters
3. Human approval for high-impact actions
4. Log every invocation and outcome

Maintenance Discipline



REGRESS

Retest after every model update



RED TEAM

Scheduled bypass simulations



ALIGN

Track policy and regulation change



SYNC

OWASP + ATLAS are living documents

1 Direct Injection

2 Jailbreak Framing

3 Data Exfiltration

4 Schema Violation

5 Low-Conf Overreliance

6 Indirect Injection

HOW TO RUN

WINDOWS

```
python guardrail_design_tester.py
```

MACOS/LINUX

```
python3 guardrail_design_tester.py
```

TRY IT Set `INPUT_INJECTION_DETECTION=False`. Cases T01 and T06 now succeed. Protection rate drops. Re-enable to see defense-in-depth restore it.

`guardrail_design_tester.py`

layered input+output guardrails tested against 6 adversarial cases

Demo: Layered Guardrail Tester

Live OpenAI API — injection, jailbreak, exfiltration, schema tests

Episode Takeaways

Capability $\neq$ permission — guardrails enforce the boundary

Input rails stop hostile instructions before the model

Output rails catch what input rails miss

Agent rails bound tools, approvals, and logging

Model Hardening and Defensive Architecture

Chapter 07 · Episode 03 · Domain 2.2 — Secure by Design

Hardening vs Patching

REACTIVE PATCHING

Fix after discovery

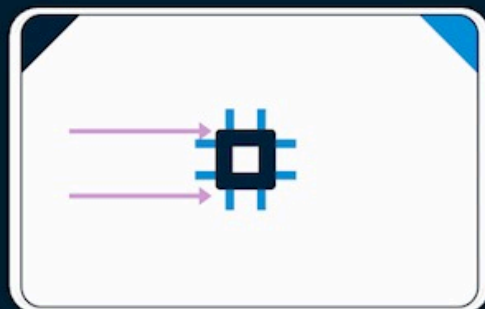
Lags the threat landscape

PROACTIVE HARDENING

Anticipate failure modes

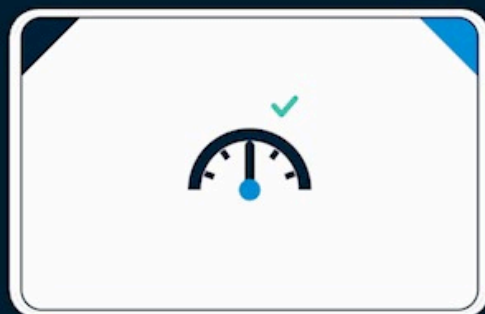
Reduce weaknesses in advance

Four Hardening Indicators



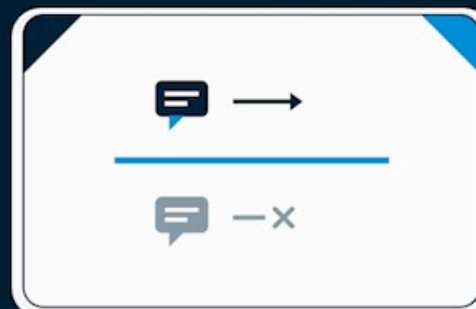
TRAIN

Adversarial examples
in training



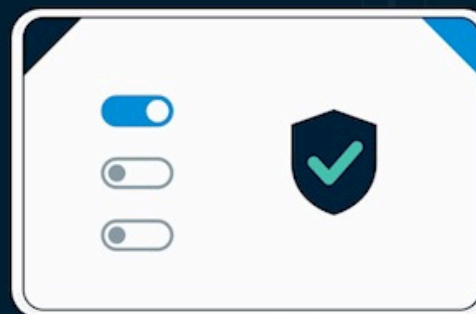
CALIBRATE

Outputs reflect
true uncertainty



THRESHOLD

Gate low-confidence responses



REDUCE

Fewer capabilities, less surface

Capability Minimization

Not needed? Not enabled
Limit modalities and outputs
Constrain context windows



Unrelated Misuse = Excess Scope

Minimal capability is a strategy
Not a limitation



Adversarial Training

TECHNIQUE

Hostile inputs during training

Stronger decision boundaries

Calibrated confidence

EXAM CONTEXT

Proactive, not post-incident

Can embed bias if mismanaged

Iterative — never one-time



Segmentation & Isolation

1. Isolate training from inference
2. Separate admin from public APIs
3. Authenticated pathways only
4. Expose only required endpoints

Hardening Is a Lifecycle

Re-evaluate after every update

Regression tests catch silent gaps

Rollback = hardening + continuity



1 Attack Surface Audit

M
O
D

2 Adversarial Training Sim

M
O
D

3 Segmentation Checker

M
O
D

HOW TO RUN

WINDOWS

```
python ep18_demo.py
```

MACOS/LINUX

```
python3 ep18_demo.py
```

TRY IT Change the robustness threshold in the adversarial training module. At what score does the audit flag a risk breach?

`ep18_demo.py`

pure Python 3.10+, no external libraries

Demo: Model Hardening Audit

Capability · training · segmentation · regression — pass/warn/fail

Exam Lens

Secure by design = proactive hardening

Unneeded functionality must be disabled

Adversarial training can embed bias

Cross-boundary cascade = segmentation failed

AI Gateway Architecture and Request Mediation

Chapter 07 · Episode 04 · Domain 2.2 — The Centralized Control Plane

Why Mediate Requests?

DIRECT EXPOSURE

No validation, logging, limits

Injection, abuse, leakage

GATEWAY-MEDIATED

Every request inspected

One enforcement boundary

Deterministic Guards, Probabilistic Model

Policy enforced outside the model

Reliable rules the model can't skip



Core Gateway Functions



AUTH

Verify caller —
scope permissions



LIMIT

Rate + quota stop exhaustion



VALIDATE

Inspect and
sanitize payloads



LOG

Full metadata — evidence trail

Volume & Modality Controls



TOKENS

Ceilings cap cost and manipulation



SIZE

Blocks hidden payload embedding



MODALITY

Disable parsers you don't need

Prompt Mediation

SYSTEM

User input never overrides

SEGMENT

KB · documents · queries apart

REWRITE

Structured templates, filtered

REJECT

Conflicting instructions denied

Injection blends in when boundaries blur





Gateway as Monitor

1. Real-time anomaly detection
2. Abuse-pattern recognition
3. SIEM and SOC correlation
4. Adaptive policy updates

Both Prevent and Detect

Blocks bad input

Generates monitoring signals

Primary defensive boundary



1 Auth & Rate Limit

M
O
D

2 Input Validation

M
O
D

3 Modality Filter

M
O
D

HOW TO RUN

WINDOWS

```
python ep19_demo.py
```

MACOS/LINUX

```
python3 ep19_demo.py
```

TRY IT Try disabling the rate limiter. How many requests does it take before extraction becomes feasible?

ep19_demo.py

pure Python 3.10+, no external libraries

Demo: Gateway Inspection Pipeline

Auth · validation · ceilings · rate · modality · logging

Exam Lens

No gateway = primary architectural gap

Gateway deterministic — model probabilistic

Extraction or DoS → rate limiting failed

Prompt Firewalls and Injection Defense

Chapter 07 · Episode 05 · Domain 2.2 — Defending Instruction Integrity

A Semantic Attack

No coding flaw exploited

Models are built to follow instructions

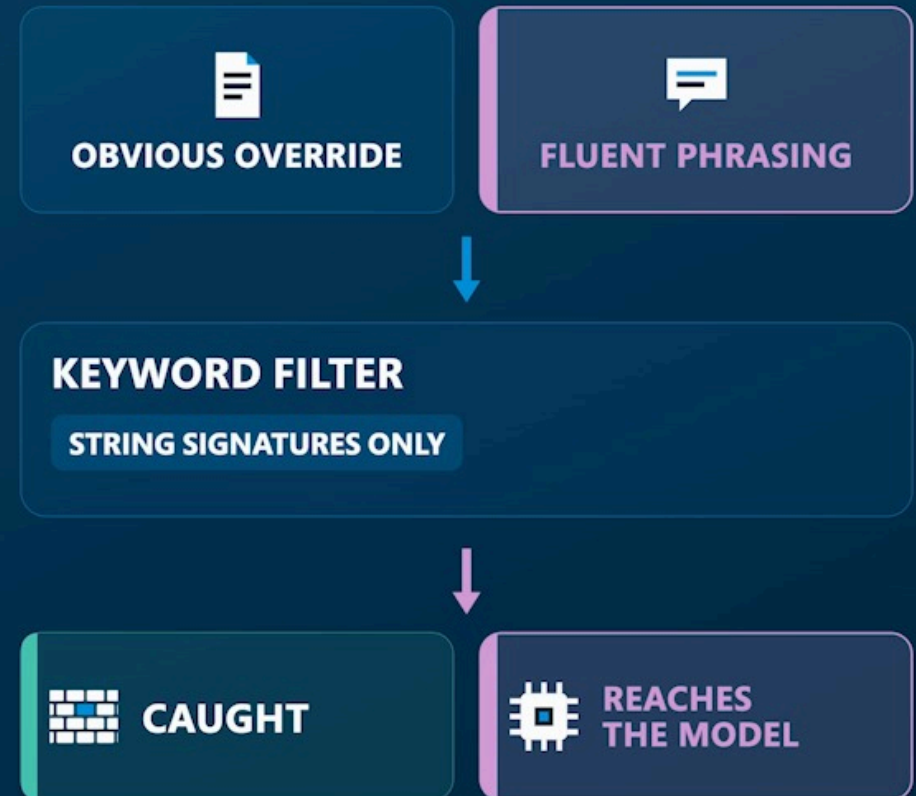
Feature becomes vulnerability



Why Keyword Filters Fail

Natural language, no signatures
Poisoned docs carry hidden orders
Defend meaning, not tokens

TWO ATTACKS · ONE FILTER



Signatures match strings. Injection is meaning — defend intent, not tokens.

Dual-Stage Firewall

PRE-INFERENCE

Inspect queries for overrides

System prompts stay authoritative

Policy validation engine

POST-INFERENCE

Verify outputs before delivery

Catch leaks and violations

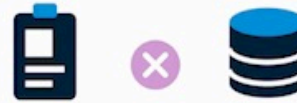
Schema and policy checks

Understand Intent, Not Words



INTENT

What is the request trying to do?



ROLE

Does the ask fit the system role?



ANOMALY

Structure odd, vocabulary benign



RISK

Score threats — route to review

RAG Poisoning Defense



SEGMENT

Isolate trusted from retrieved



LABEL

Trust-score every doc source



FILTER

Scan docs for hidden directives



Prove It Still Works

1. Red-team injection simulations
2. Bypass testing — indirect phrasing
3. Monitor injection-attempt clusters
4. Adapt filters as language evolves

1 Direct injection attempt

S
C
N

2 RAG doc poisoning

S
C
N

3 Role consistency check

S
C
N

HOW TO RUN

WINDOWS

```
python ep20_demo.py
```

MACOS/LINUX

```
python3 ep20_demo.py
```

TRY IT Craft a new injection attempt using indirect phrasing. Does the semantic classifier catch it? At what risk score?

ep20_demo.py

pure Python 3.10+, no external libraries

Demo: Live Injection Attack

GPT-4o mini — with and without a prompt firewall

Exam Lens

Semantic attack → semantic defense

Both firewall stages required

RAG needs its own filtering layer

Untested firewall = false assurance